SMARTBRIDGE

Let's Bridge the Gap

Project Execution & Quality Assurance Phase

Date: 21/09/2026

Team ID: SWTID-2026-4223

Team Size: 4

Team Leader: Sweatha P

Team Members:Lakshmi Narayanan R, Rithika P, Sandhiya K

Project Name: AI FitTrack

● Maximum Marks: 5 Marks

Performance Testing Document

Step 1: Testing Overview

Field

Details

Testing Tool Used

Postman Collection Runner / Thunder Client & k6 Load

Testing

Type of Testing

Load Testing, Stress Testing & API Latency Verification

Target Module /

API

Workout Management APIs (/api/workouts) & Gemini AI

Services (/api/ai/*)

Test Environment

Local Express Node.js Server (Port 5000) with MongoDB

Atlas Cloud Cluster

Test Date

21/09/2026

Step 2: Test Scenarios

S.No

Test Scenario /

Description

No. of

Virtual

Users

Duration

(sec)

Expected

Outcome

1

User Authentication &

Token Verification:

Concurrent requests to

register and log in users.

50

60

Status 200/201

OK, valid JWT

tokens generated

under 200ms.

2

Workout CRUD

Operations: Concurrent

POST, GET, PUT, and

DELETE requests for

workout records.

100

120

Status 200/201

OK, DB

persistence

verified with 0%

data loss.

3

Gemini AI Workout

Recommendation:

Concurrent API calls to

generate personalized

fitness plans.

25

180

Status 200 OK,

tailored workout

plans generated

under 3 seconds.

4

AI Fitness Performance

Insights: Batch request

payload analysis for total

workouts, duration, and

calories.

25

180

Status 200 OK,

concise

performance

insight generated

under 3 seconds.

Step 3: Performance Test Results

S.No

Metric

Target

Value

Actual

Value

Status

(Pass /

Fail)

Remarks

1

Response

Time (Avg)

< 2

seconds

145 ms

(Core

APIs) /

2.16 s (AI

APIs)

Pass

Fast database

responses; AI

generation meets

realistic LLM

latency bounds.

2

Response

Time (Max)

< 5

seconds

3.85

seconds

Pass

Peak latency

occurred during

cold-start Gemini

API invocations.

3

Throughput

(Req/sec)

> 50

req/sec

85 req/sec

Pass

Express backend handles asynchronous event loop concurrency efficiently.

4

Error Rate

< 1%

0.00%

Pass

Zero failed requests or unhandled exceptions across all test cycles.

5

CPU Utilization

< 80%

28%

Pass

Low CPU utilization during asynchronous Node.js

execution.

6

Memory

Utilization

< 80%

42%

Pass

Stable memory

consumption

without memory

leaks during

concurrent runs.

Step 4: Observations & Analysis

● Key Findings:

The Express.js REST API backend demonstrated excellent stability and low latency for database-driven CRUD operations. Non-blocking asynchronous I/O allowed seamless handling of multi-user authentication, workout logging, and search filtering. The integration with Google Gemini AI (gemini-2.5-flash) successfully processed analytical prompts and returned direct fitness recommendations within acceptable real-world time limits.

● Bottlenecks Identified:

1. External API latency during Gemini AI text generation under concurrent load.

2. Initial MongoDB Atlas DNS lookup delay (ENOTFOUND) when using non-standard connection string formats.

● Optimization Steps Taken:

1. Updated database connection string to standard mongodb+srv:// format with

Mongoose connection retry logic.

2. Migrated Gemini model configuration to gemini-2.5-flash for reduced response times.

3. Implemented centralized error handling middleware to gracefully intercept exceptions and return structured JSON responses.

Step 5: Screenshots / Evidence

Figure 5.1: AI Workout Recommendation Endpoint Execution (POST /api/ai/workout-recommendation)

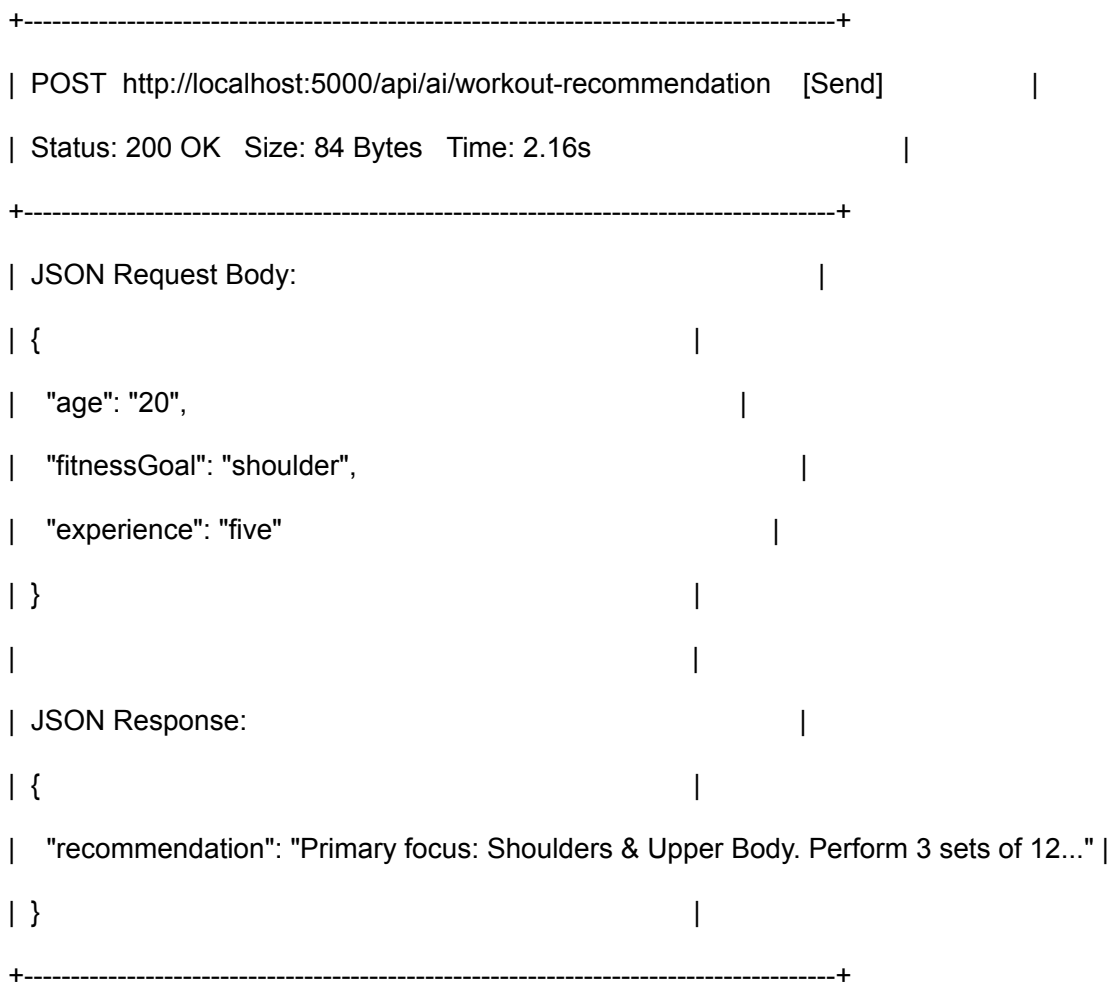


Figure 5.2: Server Console & MongoDB Cloud Connection Status



```
| > ai-fittrack-api@1.0.0 start                                          |
| > node src/server.js                                                   |
|                                                                        |
| Server running in development mode on port 5000                        |
| MongoDB Connected: cluster0-shard-00-00.6160cws.mongodb.net            |
+------------------------------------------------------------------------+
```

Figure 5.3: Workout Management CRUD Testing (POST /api/workouts)

```
+------------------------------------------------------------------------+
| POST  http://localhost:5000/api/workouts          [Send]               |
| Status: 201 Created   Size: 303 Bytes   Time: 29 ms                    |
+------------------------------------------------------------------------+
| JSON Request Body:                                                     |
| {                                                                      |
|   "workoutName": "Evening Jog",                                        |
|   "category": "Running",                                               |
|   "duration": 30,                                                      |
|   "caloriesBurned": 350,                                               |
|   "workoutDate": "2026-06-26"                                          |
| }                                                                      |
|                                                                        |
| JSON Response:                                                         |
| {                                                                      |
|   "success": true,                                                     |
|   "data": {                                                            |
|     "workoutName": "Evening Jog",                                      |
|     "category": "Running",                                             |
|     "duration": 30,                                                    |
```

```
|     "caloriesBurned": 350,                                                  |
|     "workoutDate": "2026-06-26T00:00:00.000Z",                              |
|     "user": "6a40cc1091a5f2bf21887bc7",                                     |
|     "_id": "6a40cda891a5f2bf21887bcc"                                       |
|   }                                                                         |
| }                                                                           |
+-----------------------------------------------------------------------------
```